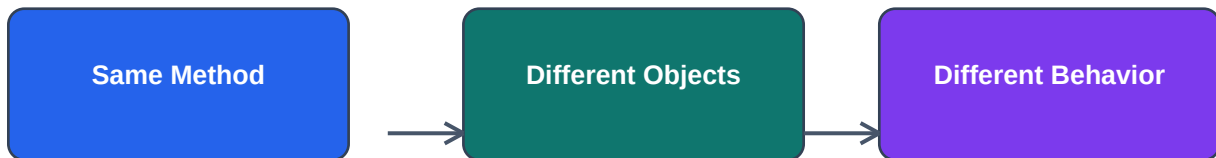


POLYMORPHISM IN PYTHON

Professional Notes with Code, Outputs, Diagrams and Real-Time Use Cases



One interface, many implementations

Prepared by Thanesh

Polymorphism means **one interface with many forms**. The same method name can perform different actions depending on the object that receives the call.

1. What is Polymorphism?

The word polymorphism comes from **poly** (many) and **morph** (forms). In programming, it allows the same operation, method, or function to behave differently for different data types or objects.

Simple idea: We call the same method, but Python selects the correct behavior at runtime.

2. Why Do We Use Polymorphism?

Benefit	Explanation
Flexibility	The same code works with many object types.
Extensibility	New child classes can be added without changing existing calling code.
Loose coupling	The caller depends on common behavior, not a specific implementation.
Maintainability	Business logic becomes cleaner and easier to test.
Reusability	Shared method names reduce repetitive conditional logic.

3. Basic Polymorphism with Built-in Functions

Python built-in functions already demonstrate polymorphism. The same **len()** function works with strings, lists, tuples, and dictionaries.

```
print(len("Python"))
```

```
print(len([10, 20, 30, 40]))
print(len((1, 2, 3)))
print(len({"name": "Thanesh", "city": "Bangalore"}))
```

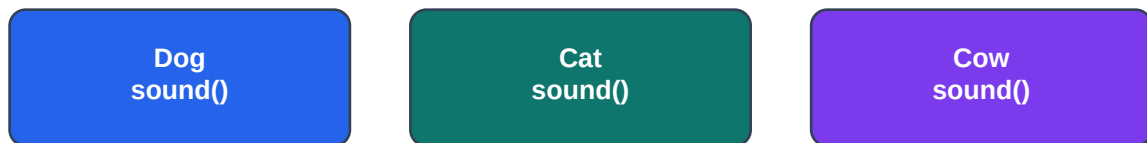
Output

```
6
4
3
2
```

Here, **len()** is one function, but it calculates length differently for each object type.

4. Polymorphism through Method Overriding

Method overriding occurs when child classes provide their own implementation of a method defined in the parent class.



Python calls sound() based on the actual object

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")

class Cow(Animal):
    def sound(self):
        print("Cow moos")

animals = [Dog(), Cat(), Cow()]

for animal in animals:
    animal.sound()
```

Output

```
Dog barks
Cat meows
Cow moos
```

The loop contains one method call: **animal.sound()**. The output changes according to the actual object stored in **animal**.

5. Polymorphism with a Common Method

Python does not require every class to share a formal parent class. If different objects provide the required method, the same code can work with all of them. This is commonly called **duck typing**.

```
class Car:
    def move(self):
        print("Car moves on the road")

class Boat:
    def move(self):
        print("Boat moves on water")

class Aeroplane:
    def move(self):
        print("Aeroplane flies in the sky")

def start_journey(vehicle):
    vehicle.move()

start_journey(Car())
start_journey(Boat())
start_journey(Aeroplane())
```

Output

```
Car moves on the road
Boat moves on water
Aeroplane flies in the sky
```

The **start_journey()** function does not check whether the object is a Car, Boat, or Aeroplane. It only expects the object to provide a **move()** method.

6. Operator Polymorphism

The same operator can behave differently for different operands. The **+** operator performs arithmetic addition for numbers and concatenation for strings or lists.

```
print(10 + 20)
print("Python " + "Programming")
print([1, 2] + [3, 4])
```

Output

```
30
Python Programming
[1, 2, 3, 4]
```

7. Custom Operator Overloading

Python allows classes to define operator behavior using special methods. For example, **__add__()** controls how the **+** operator works for objects.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __add__(self, other):
        return self.salary + other.salary

e1 = Employee("Rahul", 45000)
e2 = Employee("Priya", 55000)
```

```
print("Total Salary:", e1 + e2)
```

Output

```
Total Salary: 100000
```

When Python evaluates **e1 + e2**, it internally calls **e1.__add__(e2)**.

8. Compile-Time vs Runtime Polymorphism

Type	Meaning	Python Support
Compile-time polymorphism	Method overloading is resolved before execution	Python does not support traditional method overloading by signature
Runtime polymorphism	Overridden method is selected while the program runs	Fully supported through method overriding and duck typing.

9. Method Overloading in Python

Python does not support traditional method overloading by defining the same method multiple times. The latest definition replaces earlier ones. Similar behavior is achieved using default arguments, ***args**, or type checking.

```
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

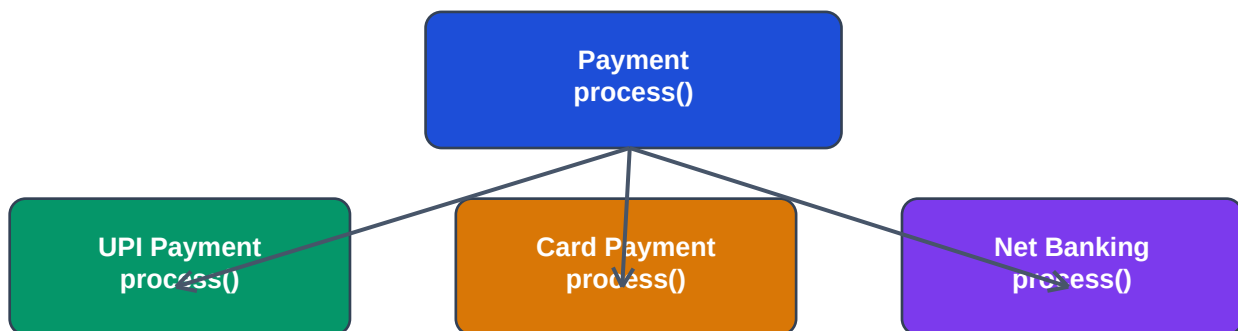
calc = Calculator()
print(calc.add(10))
print(calc.add(10, 20))
print(calc.add(10, 20, 30))
```

Output

```
10
30
60
```

10. Real-Time Use Case: Payment Processing System

A payment application may support UPI, credit cards, and net banking. The checkout service should call one common method, while each payment type performs its own validation and processing.



```
class Payment:
    def process(self, amount):
        raise NotImplementedError("Child class must implement process()")

class UPIPayment(Payment):
    def process(self, amount):
        print(f"Processing UPI payment of Rs.{amount}")
        print("UPI payment completed successfully")

class CardPayment(Payment):
    def process(self, amount):
        print(f"Processing card payment of Rs.{amount}")
        print("Card payment completed successfully")

class NetBankingPayment(Payment):
    def process(self, amount):
        print(f"Processing net banking payment of Rs.{amount}")
        print("Net banking payment completed successfully")

def checkout(payment_method, amount):
    payment_method.process(amount)

checkout(UPIPayment(), 1500)
checkout(CardPayment(), 2500)
checkout(NetBankingPayment(), 3500)
```

Why this is useful

The checkout function is not changed when a new payment type is introduced. We can add WalletPayment or CashOnDelivery by creating another class with the same **process()** method.

11. Real-Time Banking Use Case: Different Account Types

Banks calculate interest and withdrawal rules differently for savings and current accounts. Polymorphism allows one banking service to handle both account types through common methods.

```
class BankAccount:
    def __init__(self, account_no, holder_name, balance):
        self.account_no = account_no
        self.holder_name = holder_name
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
```

```

        self.balance += amount
        print(f"Rs.{amount} deposited successfully")
    else:
        print("Deposit amount must be greater than zero")

def withdraw(self, amount):
    raise NotImplementedError("Child class must implement withdraw()")

def calculate_interest(self):
    raise NotImplementedError("Child class must implement calculate_interest()")

def display(self):
    print("Account Number:", self.account_no)
    print("Holder Name    :", self.holder_name)
    print("Balance        :", self.balance)

class SavingsAccount(BankAccount):
    def withdraw(self, amount):
        minimum_balance = 1000

        if amount <= 0:
            print("Withdrawal amount must be greater than zero")
        elif self.balance - amount >= minimum_balance:
            self.balance -= amount
            print(f"Rs.{amount} withdrawn from Savings Account")
        else:
            print("Minimum balance of Rs.1000 must be maintained")

    def calculate_interest(self):
        interest = self.balance * 0.04
        print("Savings interest:", interest)
        return interest

class CurrentAccount(BankAccount):
    def withdraw(self, amount):
        overdraft_limit = 5000

        if amount <= 0:
            print("Withdrawal amount must be greater than zero")
        elif amount <= self.balance + overdraft_limit:
            self.balance -= amount
            print(f"Rs.{amount} withdrawn from Current Account")
        else:
            print("Overdraft limit exceeded")

    def calculate_interest(self):
        print("Current Account does not earn interest")
        return 0

def account_summary(account):
    account.display()
    account.calculate_interest()
    print("-" * 35)

savings = SavingsAccount("SB101", "Thanesh", 20000)
current = CurrentAccount("CA201", "ABC Technologies", 5000)

accounts = [savings, current]

for account in accounts:
    account_summary(account)

```

```
print("Performing Withdrawals")
savings.withdraw(3000)
current.withdraw(8000)
```

Key polymorphic calls

account.calculate_interest() and **account.withdraw()** produce different results depending on whether the object is a SavingsAccount or CurrentAccount.

12. Menu-Driven Banking Application Using Polymorphism

The following program combines inheritance, method overriding, polymorphism, validation, and a menu-driven interface.

```
class BankAccount:
    def __init__(self, account_no, holder_name, balance):
        self.account_no = account_no
        self.holder_name = holder_name
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print("Amount deposited successfully")
        else:
            print("Invalid deposit amount")

    def withdraw(self, amount):
        raise NotImplementedError

    def calculate_interest(self):
        raise NotImplementedError

    def check_balance(self):
        print("Current Balance:", self.balance)

    def display_details(self):
        print("\n----- Account Details -----")
        print("Account Number:", self.account_no)
        print("Holder Name    :", self.holder_name)
        print("Account Type   :", self.__class__.__name__)
        print("Balance        :", self.balance)

class SavingsAccount(BankAccount):
    def withdraw(self, amount):
        minimum_balance = 1000
        if amount <= 0:
            print("Invalid withdrawal amount")
        elif self.balance - amount >= minimum_balance:
            self.balance -= amount
            print("Withdrawal successful")
        else:
            print("Cannot withdraw: maintain Rs.1000 minimum balance")

    def calculate_interest(self):
        interest = self.balance * 4 / 100
        self.balance += interest
        print("4% interest added:", interest)

class CurrentAccount(BankAccount):
    def withdraw(self, amount):
        overdraft_limit = 5000
        if amount <= 0:
            print("Invalid withdrawal amount")
        elif amount <= self.balance + overdraft_limit:
            self.balance -= amount
            print("Withdrawal successful using current account")
        else:
            print("Overdraft limit exceeded")

    def calculate_interest(self):
```



```

        print("No interest is provided for Current Account")

def create_account():
    print("\n1. Savings Account")
    print("2. Current Account")
    account_type = input("Choose account type: ")

    account_no = input("Enter account number: ")
    holder_name = input("Enter account holder name: ")
    balance = float(input("Enter initial balance: "))

    if account_type == "1":
        return SavingsAccount(account_no, holder_name, balance)
    elif account_type == "2":
        return CurrentAccount(account_no, holder_name, balance)
    else:
        print("Invalid account type")
        return None

account = create_account()

if account is not None:
    while True:
        print("\n===== BANK MENU =====")
        print("1. Deposit")
        print("2. Withdraw")
        print("3. Check Balance")
        print("4. Calculate/Add Interest")
        print("5. Display Account Details")
        print("6. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            amount = float(input("Enter deposit amount: "))
            account.deposit(amount)

        elif choice == "2":
            amount = float(input("Enter withdrawal amount: "))
            account.withdraw(amount)

        elif choice == "3":
            account.check_balance()

        elif choice == "4":
            account.calculate_interest()

        elif choice == "5":
            account.display_details()

        elif choice == "6":
            print("Thank you for using the banking application")
            break

        else:
            print("Invalid choice. Please try again")

```

How polymorphism works in this program

Step	Explanation
1	create_account() returns either a SavingsAccount or CurrentAccount object.

Step	Explanation
2	The variable <code>account</code> can refer to either child object.
3	<code>account.withdraw()</code> calls the correct overridden method at runtime.
4	<code>account.calculate_interest()</code> also behaves according to the selected account type.
5	The menu code remains unchanged for both account types.

13. Real-Time Applications of Polymorphism

Domain	Common Interface	Different Implementations
Banking	<code>withdraw()</code> , <code>calculate_interest()</code>	Savings, Current, Salary, Loan accounts
Payments	<code>process()</code>	UPI, Card, Wallet, Net Banking
Notifications	<code>send()</code>	Email, SMS, WhatsApp, Push notification
E-commerce	<code>calculate_price()</code>	Regular, Premium, Festival discount
Cloud storage	<code>upload()</code>	AWS S3, Azure Blob, Google Cloud Storage
Reports	<code>generate()</code>	PDF, Excel, CSV, HTML
Vehicles	<code>start()</code> , <code>move()</code>	Car, Bike, Bus, Electric Vehicle
Authentication	<code>authenticate()</code>	Password, OTP, Google login, Biometric

14. Polymorphism vs Inheritance

Inheritance	Polymorphism
Creates an IS-A relationship.	Allows one interface to have many implementations.
Focuses on acquiring parent members.	Focuses on different behavior for the same call.
Implemented using parent and child classes.	Implemented using overriding, duck typing, and operator overloading.
Example: <code>SavingsAccount</code> inherits <code>BankAccount</code> .	Example: <code>withdraw()</code> behaves differently for each account type.

15. Best Practices and Common Mistakes

Best practices

- Use a meaningful common interface.
- Keep method names and parameter meanings consistent.
- Use abstract base classes when implementations must follow a contract.
- Prefer polymorphism over long if-elif chains based on object type.
- Validate data inside each concrete implementation.

Common mistakes

- Defining the same method repeatedly and expecting traditional overloading.
- Calling a method that some objects do not implement.
- Giving overridden methods incompatible parameters.

- Using inheritance only to reuse code when there is no logical IS-A relationship.
- Creating too many child classes for very small behavior differences.

16. Interview Questions and Answers

What is polymorphism?

Polymorphism allows the same method, function, or operator to behave differently for different objects.

How is runtime polymorphism achieved in Python?

It is mainly achieved through method overriding and duck typing.

Does Python support method overloading?

Python does not support traditional signature-based overloading. Similar behavior is implemented using default arguments, *args, or singledispatch.

What is duck typing?

Python focuses on whether an object provides the required behavior rather than checking its exact class.

What is operator overloading?

It is customizing operators for user-defined objects using special methods such as `__add__()`, `__lt__()`, and `__eq__()`.

Give a real-time example.

A payment service calls `process()` for UPI, card, wallet, or net banking objects, and each object performs its own payment logic.

17. One-Minute Interview Answer

Polymorphism is an OOP concept that allows one method, function, or interface to have multiple implementations. In Python, it is commonly achieved using method overriding, duck typing, built-in function behavior, and operator overloading. For example, a banking application can call the same `withdraw()` method on savings and current account objects, but each account applies different withdrawal rules. Polymorphism improves flexibility, extensibility, loose coupling, and maintainability.

18. Practice Exercises

1. Create Employee, Developer, and Trainer classes with a `calculate_salary()` method.
2. Build EmailNotification, SMSNotification, and WhatsAppNotification classes with `send()`.
3. Create Circle, Rectangle, and Triangle classes with `area()`.
4. Implement FoodOrder payment using UPI, card, and cash classes.
5. Extend the banking menu with a SalaryAccount class and separate interest rules.